Kathará

# FRRouting

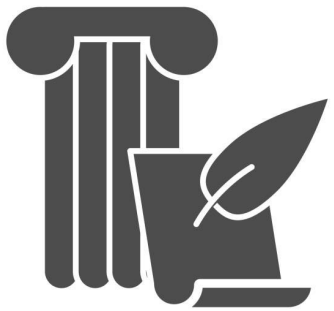
© Computer Networks Research Group
Roma Tre

# Summary

- What is routing?
- Kathara Implementation (lab on FRR)

© Computer Networks Research Group
Roma Tre

# The network layer is the Inter-Net layer



- At the beginning: networking technologies were
  - **Tied to a specific transmission medium** (cable, ether)
  - Very **diverse** (many different technologies and protocols, competing for a market)
  - Aiming at **connecting few local end-user devices ->local nets (or «subnets»)**
  - **How did the architecture of the network looked like, at that time?**

# Some drawing…

# The problem:

- How to interconnect devices from different local nets, without creating a single big, local net (of the same technology)??

# Solution (well, one of the possible solutions)

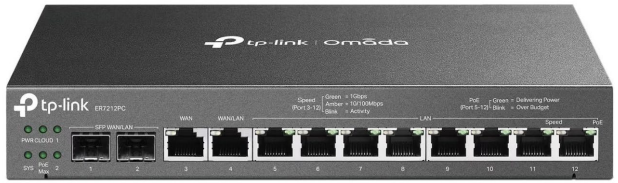
- Put all these local net technologies into a layer (the **link layer**)
- **Create a new layer on top of it**, whose task is to **deliver a message between any couple of end-user devices** from any subnet to any subnet in the world
- This layer has **its own Interface names** (IP addresses) and **intermediate systems** (called routers), just like any other subnet technology
- **Internet was born!**

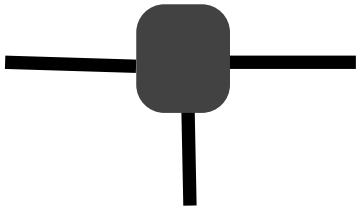
# How does the network layer work?

- To answer, we look at the main functions it implements:
  - Forwarding
  - Routing

- Let's start from the Forwarding function.

- And let us consider a single router:



# What is Forwarding?

- **Local process** that takes place **at a router** and **determines the output link for each packet**
  - Done **individually**, for each packet

- Main ingredients:

1. Information on the packet's network layer header (an identifier)
2. **Forwarding tables** at each router

# **Routing**: How to populate forwarding tables (automatically)?

Through **a routing process** that, given the network topology:

1. It computes the path from every possible source to every possible destination
2. It configures accordingly the forwarding table at each router

© Computer Networks Research Group
Roma Tre

# Routing

- Network-wide process that populates forwarding tables
- How: routing algorithm run on a logically centralised network controller or the routers themselves

# Routing: Populate forwarding tables based on knowledge of paths

- But… just any path between two subnets? Or the best?
- Best in which sense? Which metric?

# Metrics

- **Static metric: Not dependent on network status**; For example:
  - Number of nodes crossed ("hops")
  - Link capability and static delay
  - Cost
- **Dynamic metric: depends on the state of the network**
  - Link load
  - Current Packet Delay

© Computer Networks Research Group
Roma Tre

# Simple routing methods
# How routing protocols work

- **Static configuration**
- only for very simple networks
- **Flooding**
  - each duplicate packet on each outbound link; loops prevented by packet ID or other mechanisms; Duplicate packets can be received at their destination
  - Simple and robust
    - No need for routing tables
    - Rugged: Tolerates link or router failures
    - optimal in a way
    - The first package found the shortest route to the destination
  - Expensive
    - Lots of duplicate packets – little useful traffic
  - used as an ingredient by ad-hoc routing methods for mobile devices (AODV, OLSR)
- **Source routing**
  - source writes the path to the packet header
  - The router reads the next hop from the packet header, moves the pointer
  - Path discovered via flooding

© Computer Networks Research Group
Roma Tre

# Other methods

- **Distance vector** (Bellman-Ford)
  - Routers only know their local state
  - Link metrics and neighbor estimates
  - Internal routing protocols (RIP, IGRP)
- **Link state**
  - Built on global state knowledge
  - Topology database
  - Global Optimization (Shortest Path First - Dijkstra)
  - Interior Routing Protocols (OSPF)
- **Path vector**
  - No knowledge of the global status
  - path: sequence of administrative domains with attributes
  - Global optimization and policy routing
  - Exterior routing protocols (BGP)

© Computer Networks Research Group
Roma Tre

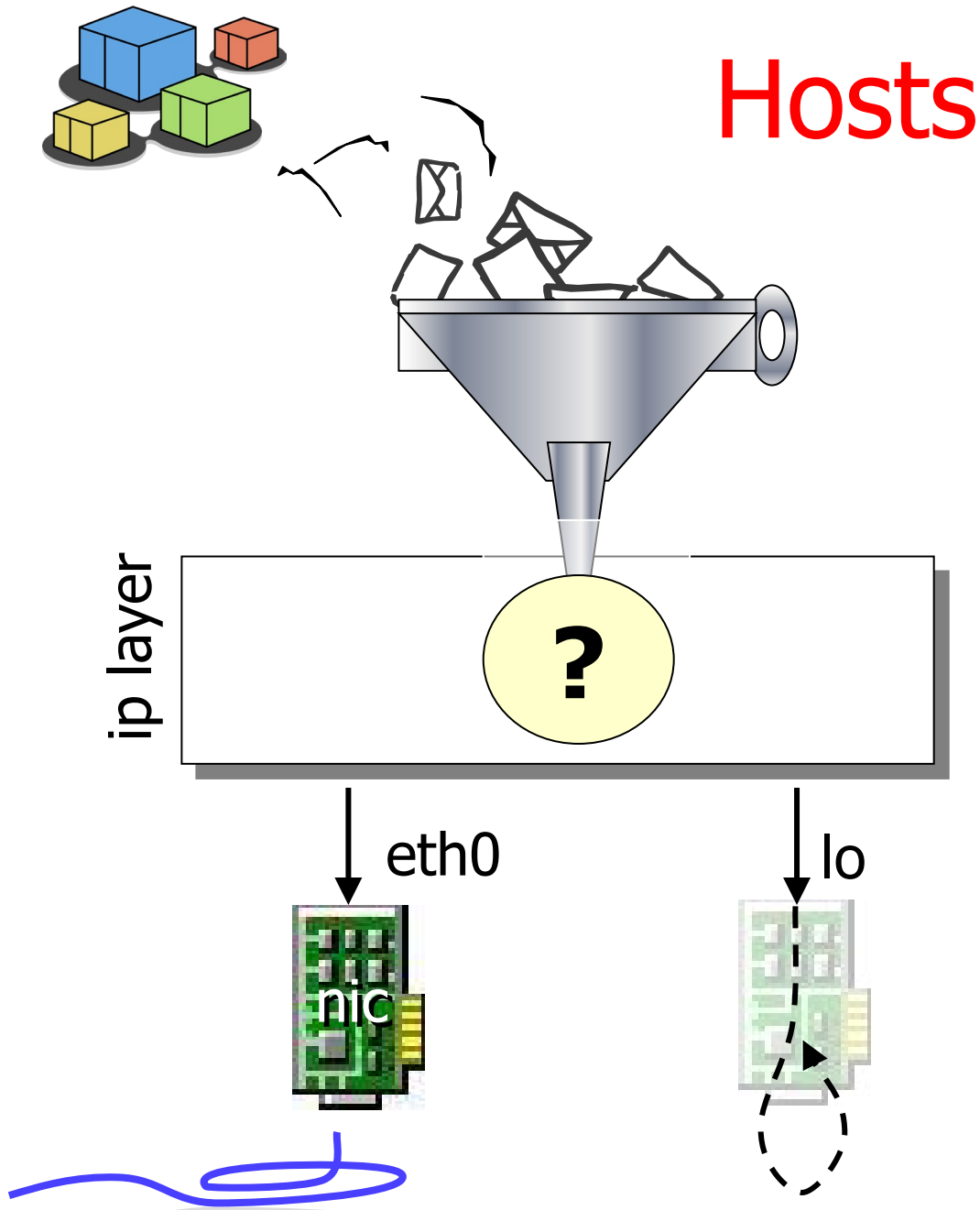
# Kathara Implementation (lab on FRR)

© Computer Networks Research Group
Roma Tre

# preconditions

- for this lab we assume you have chosen “kathara/frr” as the default image of your Kathará installation
  - execute “kathara settings”
    - select “choose default image”
    - select “kathara/frr”
    - exit from the settings procedure

# Hosts need routing




- each host with a network stack performs some elementary routing
- at the very least, the network stack may be used to access local services (e.g., Xorg)
- the host must decide when a packet needs to be sent to the network interface card (nic) and when it needs to be bounced to the loopback interface (lo)

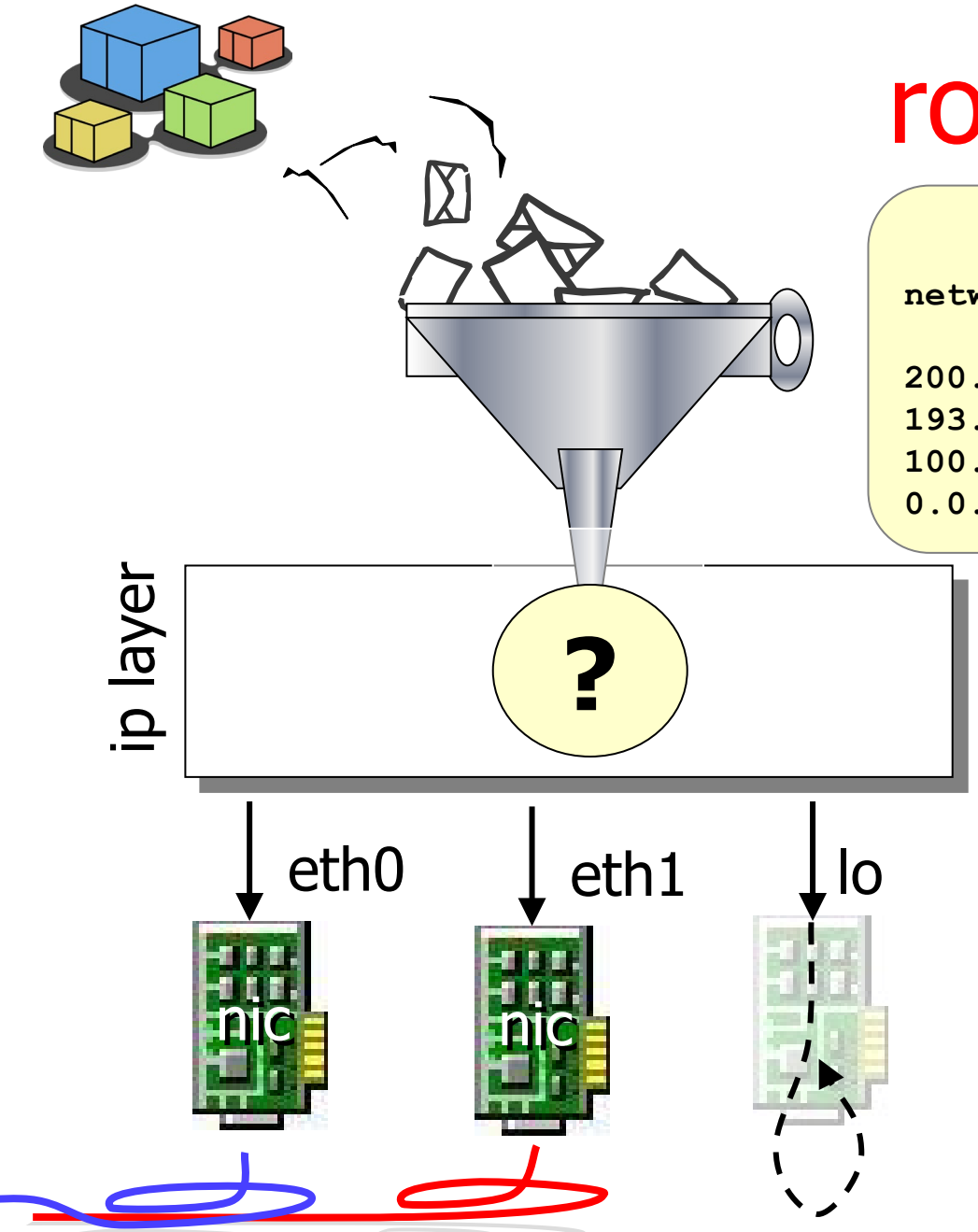
© Computer Networks Research Group Roma Tre
© Computer Networks Research Group Roma Tre

# Questions

- Is it ok to have hosts participate in routing algorithm together with routers?

© Computer Networks Research Group
Roma Tre

# routing table

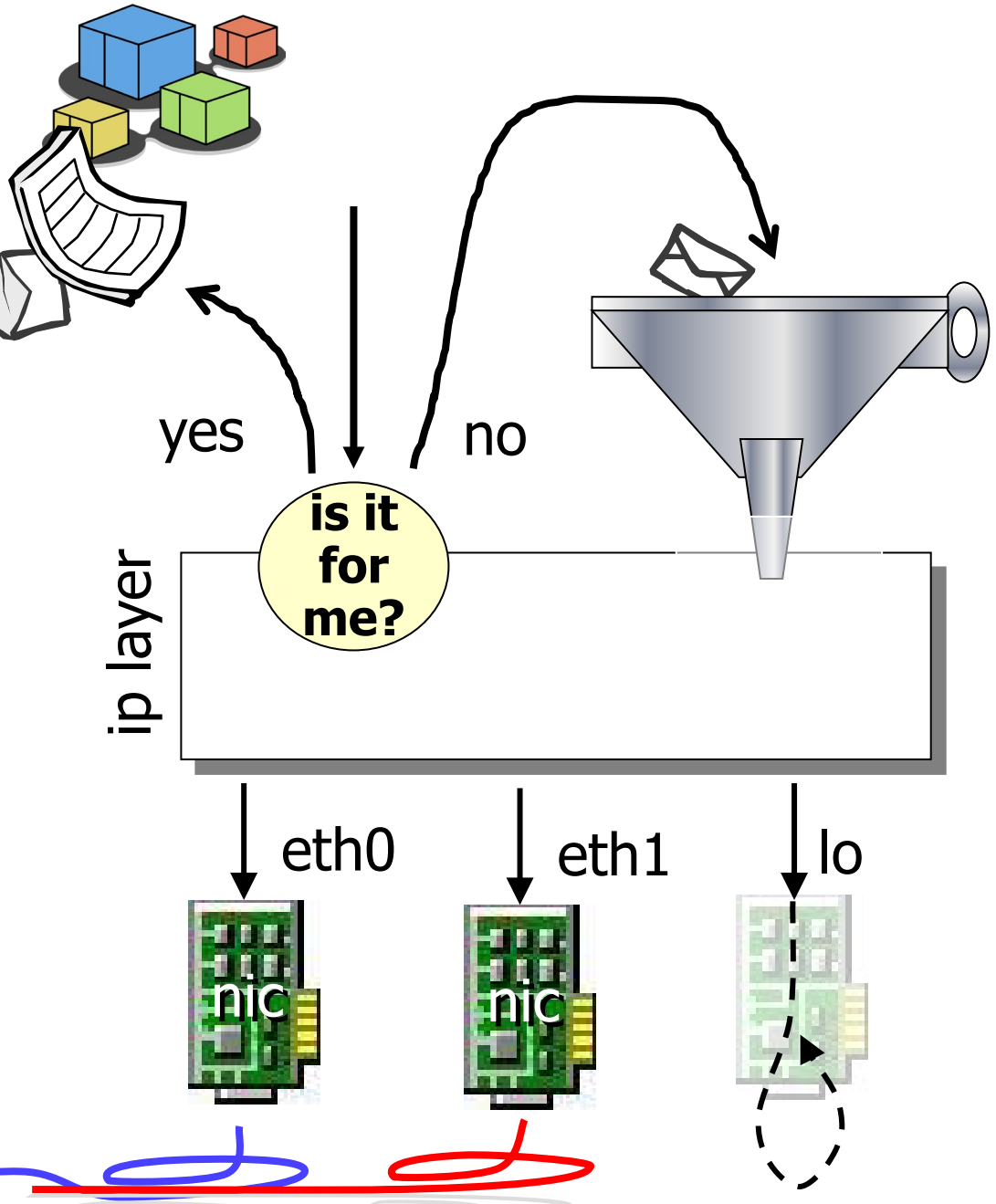



**routing table**

| network | netmask | nexthop | dev |
|---|---|---|---|
| 200.3.24.0 | 255.255.255.0 | 12.0.0.4 | eth1 |
| 193.2.0.0 | 255.255.248.0 | 11.0.0.2 | eth0 |
| 100.4.5.0 | 255.240.0.0 | 11.0.0.3 | eth0 |
| 0.0.0.0 | 0.0.0.0 | 11.0.0.2 | eth0 |

the ip layer uses a routing table to decide which is the interface the packet needs to be forwarded to

© Computer Networks Research Group
Roma Tre




# routers

- a router (also called gateway or intermediate-system)
  - has more than one network interface card
  - feeds incoming ip packets (that are not for the router itself) back in the routing process
    - this operation is called relaying or forwarding

© Computer Networks Research Group
Roma Tre

# routing protocols

- routing protocols are used to automatically update routing tables, relieving administrators from the need to do it manually
- routers (i.e., devices that run routing protocols) in Kathará are virtual machines that run a specific piece of software that implements routing protocols

**FRRouting**

© Computer Networks Research Group
Roma Tre

# The FRRouting Suite

a free and open-source Internet routing protocol suite for Linux and Unix platforms

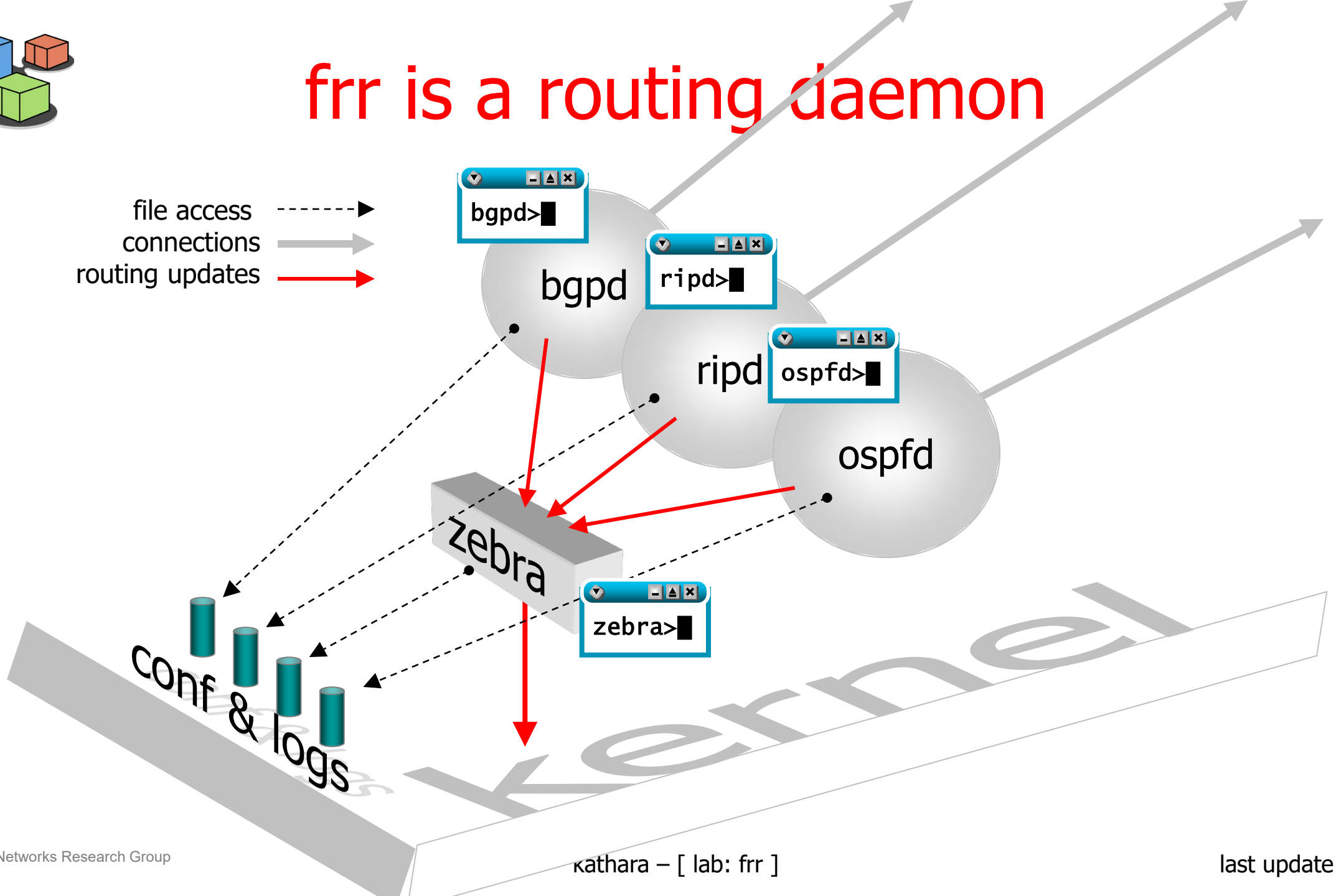
© Computer Networks Research Group
Roma Tre

# about the FRRouting Suite

- FRR is a software that implements some routing protocols
  - rip (v1 and v2), ospf (v2 and v3), is-is
  - bgp
  - pim, ldp, bfd, babel, pbr, openfabric, vrrp, …
- frr has its roots in the quagga project
  - quagga is no longer supported
- in turn, quagga was a community project that originated from (and superseded) the zebra project

© Computer Networks Research Group
Roma Tre

# frr is a routing daemon




© Computer Networks Research Group
Roma Tre

# sample **daemons** configuration file

```
root@r1:~$ cat /etc/frr/daemons
# This file tells the frr package which daemons to start.
#
# Sample configurations for these daemons can be found in
# /usr/share/doc/frr/examples/.
#
# ...
#
bgpd=no
ospfd=no
ospf6d=no
ripd=yes
ripngd=no
...
```

the rip daemon will be started

© Computer Networks Research Group
Roma Tre

# sample `vtysh.conf` configuration file

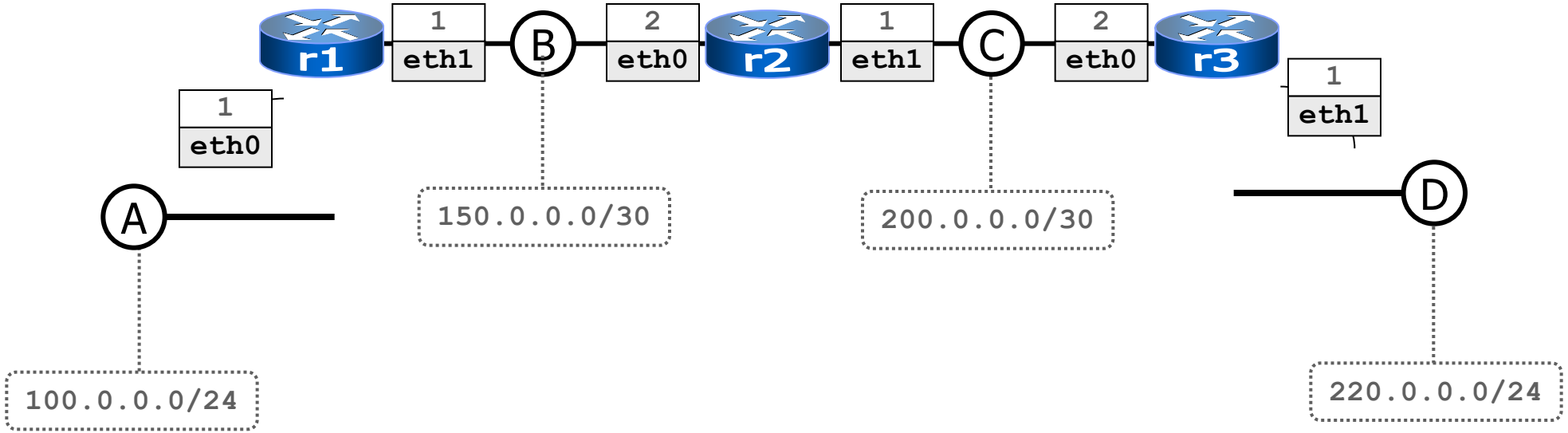
```
root@r1:~$ cat /etc/frr/vtysh.conf
no service integrated-vtysh-config

root@r1:~$
root@r1:~$
root@r1:~$
root@r1:~$
root@r1:~$
root@r1:~$ cat /etc/frr/vtysh.conf
service integrated-vtysh-config
```

with this command each daemon will need its own configuration file (default)

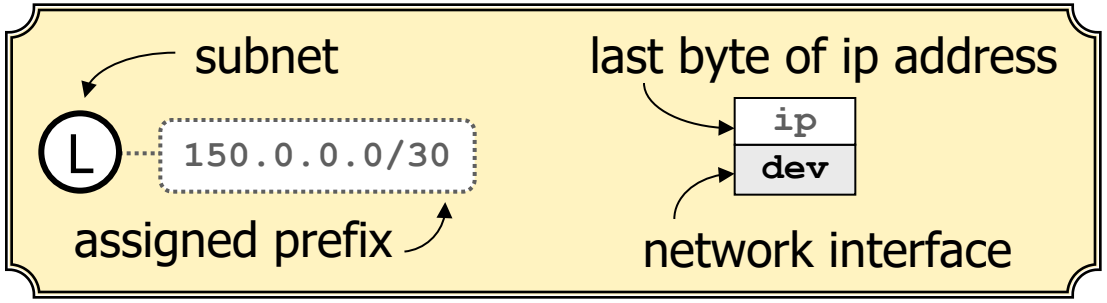
a single configuration `frr.conf` file for all daemons (recommended)

© Computer Networks Research Group
Roma Tre

# a simple topology




legend




© Computer Networks Research Group
Roma Tre

# launching the lab script

- the lab configuration is such that
  - three virtual hosts are created and connected to the right collision domains (virtual hubs)
  - for each virtual host
    - network interfaces are automatically configured
    - frr configuration files are updated
  - the frr routing daemon is automatically started

```
user@localhost:~$ cd kathara-lab_frr
user@localhost:~/kathara-lab_frr$ kathara lstart
```

© Computer Networks Research Group
Roma Tre

# automatically start frr

- to ease the startup of the lab is better to automatically start the frr
  - the main daemon and all the routing protocols daemons are automatically started with frr

r1.startup

```
systemctl start frr
```

# a one-fits-all shell

- instead of having to connect to each single daemon, users can interact with frr by using a built-in shell, called **`vtysh`**

```
root@r1:~$ vtysh
Hello, this is FRRouting (version 9.0.1).
Copyright 1996-2005 Kunihiro Ishiguro, et al.

r1#
```

- the user is not prompted for a password
- all the commands to manage all the routing daemons (including zebra itself) are available in this shell

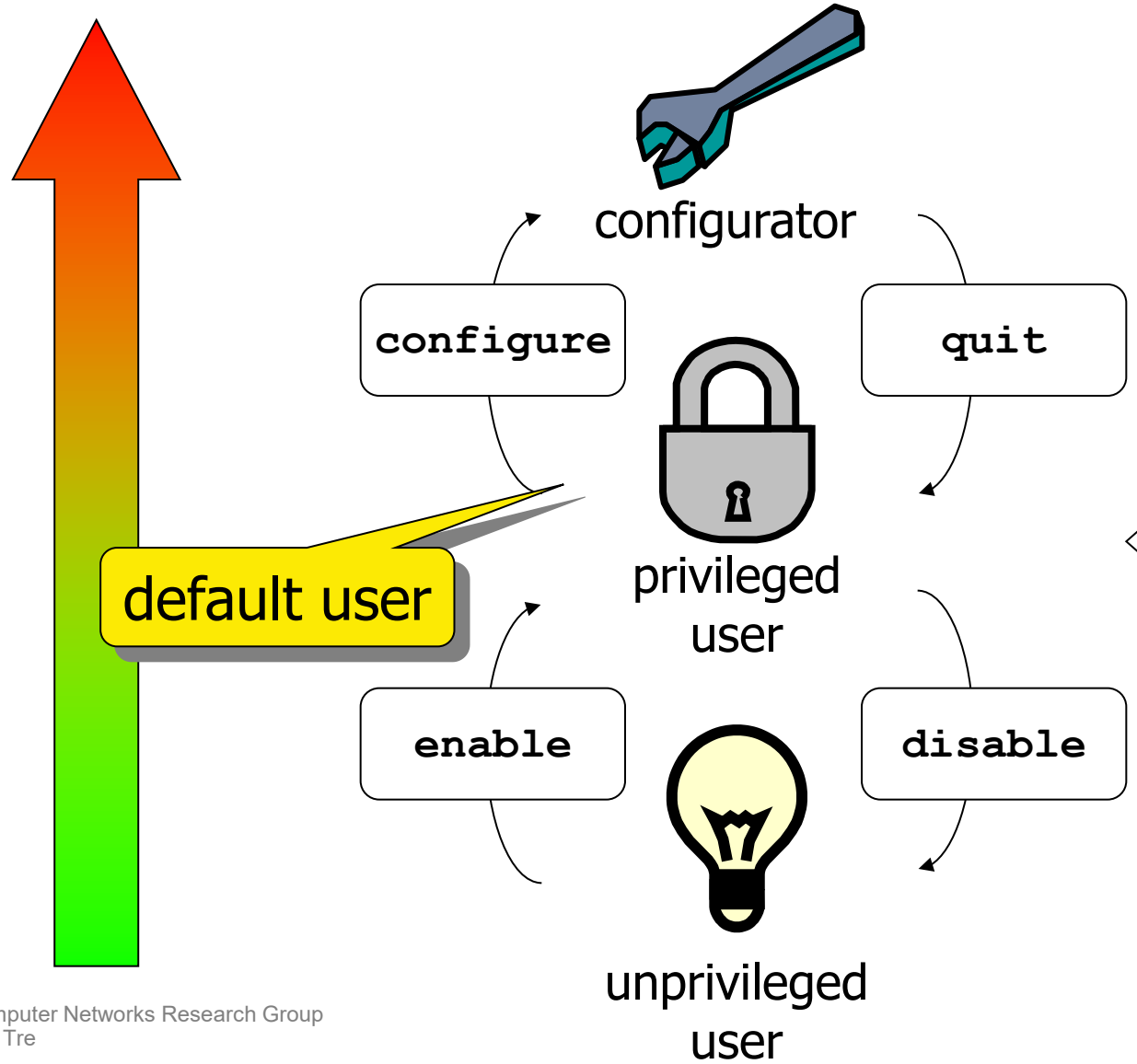
# available vtysh commands

- press '?' at the command prompt to retreive all commands available
  - here are displayed only the main ones

```
r1# ?
  configure   Configuration from vty interface
  enable      Turn on privileged mode command
  exit        Exit current mode and down to previous mode
  no          Negate a command or set its defaults
  ping        Send echo messages
  quit        Exit current mode and down to previous mode
  show        Show running system information
  terminal    Set terminal line parameters
  write       Write running configuration to memory, network, or terminal
```

© Computer Networks Research Group
Roma Tre

# privileges on a router




- performs persistent configurations affecting the router behavior

- sees critical information
- performs elementary configurations that do not affect router behavior (e.g., set time and date)
- commits changes

- sees noncritical information

© Computer Networks Research Group
Roma Tre

# inspecting the frr routing table

```
r1# show ip route
  Codes: K - kernel route, C - connected, S - static, R - RIP, O - OSPF,
     I - IS-IS, B - BGP, E - EIGRP, N - NHRP, T - Table, v - VNC,
     V - VNC-Direct, A - Babel, F - PBR, f - OpenFabric,
     > - selected route, * - FIB route, q - queued, r - rejected, b - backup
     t - trapped, o - offload failure

R>* 100.0.0.0/24 [120/3] via 200.0.0.1, eth0, weight 1, 00:01:32
R>* 150.0.0.0/30 [120/2] via 200.0.0.1, eth0, weight 1, 00:01:32
C>* 200.0.0.0/30 is directly connected, eth0, 00:01:33
C>* 220.0.0.0/24 is directly connected, eth1, 00:01:33
```

- FIB entries from this table (marked with a '>') are injected into the kernel routing table


© Computer Networks Research Group Roma Tre

# inspecting the current configuration

```
r1# show running-config
Building configuration...

Current configuration:
!
frr version 9.0.1
frr defaults traditional
hostname r3
no ipv6 forwarding
no service integrated-vtysh-config
!
end
r1#
```

# some observations

- the frr configuration language is “operational” and not “declarative”
  - the configuration file is simply a list of configuration commands that would bring the router to its operative state
  - it is not an abstract description of how the router should behave according to the configurator
- you cannot remove a configuration command
  - if the command has a certain string (e.g., “ipv6 forwarding”) in order to remove it you should issue the same string preceded by “no” (e.g., “no ipv6 forwarding”)

# copyright notice


- All the pages/slides in this presentation, including but not limited to, images, photos, animations, videos, sounds, music, and text (hereby referred to as “material”) are protected by copyright.
- This material, with the exception of some multimedia elements licensed by other organizations, is property of the authors and/or organizations appearing in the first slide.
- This material, or its parts, can be reproduced and used for didactical purposes within universities and schools, provided that this happens for non-profit purposes.
- Information contained in this material cannot be used within network design projects or other products of any kind.
- Any other use is prohibited, unless explicitly authorized by the authors on the basis of an explicit agreement.
- The authors assume no responsibility about this material and provide this material “as is”, with no implicit or explicit warranty about the correctness and completeness of its contents, which may be subject to changes.
- This copyright notice must always be redistributed together with the material, or its portions.